

Centro Universitário Senac-RS

ADS - Análise e Desenvolvimento de Sistemas / SPI - Sistemas para Internet

Unidade Curricular: Qualidade de Software

Prof.: Luciano Zanuz

Atividade PBL – Aula 10

Testes Funcionais Automatizados – LocalEats

Contexto

Após evoluir a qualidade do sistema LocalEats com:

- Planejamento de testes
- Testes manuais
- Testes unitários automatizados com TDD

A equipe de Qualidade agora precisa dar o próximo passo:

Automatizar testes funcionais do sistema completo (E2E)

Até agora:

- As regras de negócio foram testadas isoladamente
- A interface do sistema ainda depende de validação manual

Agora, o desafio é:

Garantir que os fluxos principais do sistema funcionem automaticamente do ponto de vista do usuário

O sistema ainda apresenta problemas como:

- Fluxos quebrando após mudanças no frontend
- Dificuldade em validar rapidamente funcionalidades completas
- Dependência de testes manuais repetitivos
- Falta de confiança em deploys

A equipe precisa garantir que:

“Os principais fluxos do sistema funcionem corretamente de ponta a ponta.”

Para isso, será adotado:

- Testes funcionais automatizados
- Automação com Playwright
- Execução com Pytest
- Organização com Page Object Model (POM)

👉 Link para o sistema LocalEats:
<https://local-eats-unisenac.vercel.app/>

Objetivo da Atividade

Aplicar, de forma prática e orientada ao uso real:

- Automação de testes funcionais (E2E)
- Interação com interface web (cliques, inputs, navegação)
- Uso do Playwright com Python
- Geração inicial de testes com Codegen
- Refatoração com boas práticas (Page Object Model)
- Análise de fragilidade e manutenção de testes

Importante

- Foco em **testar a aplicação via interface (frontend)**
- Não implementar lógica de negócio (já feito na aula anterior)
- Utilizar:
 - Python
 - Playwright
 - Pytest
- Codegen é obrigatório como ponto de partida
- Refatoração é obrigatória (não entregar código bruto do Codegen)

👉 O mais importante é:

Compreender como testes automatizados funcionais são construídos, organizados e mantidos

Tarefas

Elaborem um documento contendo:

◆ **1. Fluxo funcional escolhido**

Selecionar **um fluxo do sistema por integrante do grupo.**

👉 Cada integrante deverá trabalhar com um fluxo diferente.

👉 Utilizar preferencialmente um dos fluxos abaixo:

🔑 1. Login de usuário

🔍 O que faz

Permite autenticar um usuário no sistema

! Problema que resolve

Garante acesso seguro às funcionalidades

🎯 Importância

Fluxo crítico de entrada no sistema

📋 Cenários esperados

- Login válido → sucesso
- Login inválido → mensagem de erro
- Campos vazios → validação

🍴 2. Navegação e visualização de restaurantes

🔍 O que faz

Permite navegar pela lista de restaurantes

! Problema que resolve

Garante que o usuário consiga explorar opções

🎯 Importância

Base da experiência do usuário

📋 Cenários esperados

- Lista carregada corretamente
- Clique em restaurante abre detalhes

📄 3. Visualização de detalhes de um restaurante

🔍 O que faz

Exibe cardápio e informações

🎯 Importância

Etapas essenciais antes da compra

📋 Cenários esperados

- Nome do restaurante visível
- Itens do cardápio exibidos

4. Adição de item ao carrinho

 O que faz
Adiciona produtos ao carrinho

 Importância
Parte central do fluxo de compra

 Cenários esperados

- Item adicionado com sucesso
- Carrinho atualizado

5. Fluxo de pedido (checkout)

 O que faz
Finaliza o pedido

 Importância
Fluxo crítico de negócio

 Cenários esperados

- Pedido finalizado com sucesso
- Feedback ao usuário

◆ **2. Teste automatizado com Codegen**

Utilizar o comando:

```
playwright codegen https://local-eats-unisenac.vercel.app/
```

 Registrar:

- Código gerado automaticamente
- Fluxo gravado
- Observações iniciais

 Explicar:

- O que o Codegen fez bem?
- O que gerou código desnecessário?

◆ **3. Implementação do teste com Pytest**

Criar um teste funcional automatizado:

👉 Estrutura mínima:

```
tests/  
    test_fluxo_X.py
```

👉 O teste deve conter:

- Nome descritivo
- Fluxo completo
- Assertions relevantes

💻 Exemplo (simplificado):

```
def test_login_com_sucesso(page):  
    page.goto("https://local-eats-unisenac.vercel.app/")  
  
    page.get_by_text("Login").click()  
    page.get_by_label("Email").fill("teste@email.com")  
    page.get_by_label("Senha").fill("123456")  
    page.get_by_text("Entrar").click()  
  
    assert page.get_by_text("Bem-vindo").is_visible()
```

◆ 4. Refatoração com Page Object Model (POM)

Refatorar o teste aplicando boas práticas.

👉 Criar estrutura:

```
pages/  
    login_page.py  
tests/  
    test_login.py
```

👉 Exemplo:

```
class LoginPage:  
    def __init__(self, page):  
        self.page = page  
  
    def acessar(self):  
        self.page.goto("https://local-eats-unisenac.vercel.app/")  
  
    def realizar_login(self, email, senha):  
        self.page.get_by_label("Email").fill(email)  
        self.page.get_by_label("Senha").fill(senha)  
        self.page.get_by_text("Entrar").click()
```

👉 O teste deve usar a página:

```
def test_login(page):  
    login = LoginPage(page)  
    login.acessar()  
    login.realizar_login("teste@email.com", "123456")  
  
    assert page.get_by_text("Bem-vindo").is_visible()
```

◆ 5. Execução dos testes

Executar:

pytest

👉 Informar:

- Total de testes
- Quantos passaram
- Quantos falharam

👉 Registrar:

- Evidência (print ou log)

◆ 6. Análise crítica dos testes

Responder:

- O teste quebrou em algum momento? Por quê?
- Quais seletores foram mais difíceis?
- O Codegen ajudou ou gerou problemas?
- O teste é confiável? Por quê?
- O que tornaria o teste mais robusto?
- Quais são os riscos de manutenção?

◆ 7. Reflexão no contexto do LocalEats

Responder:

- Testes automatizados substituem testes manuais?
- Vale a pena automatizar todos os fluxos?
- Qual tipo de teste deve ser priorizado?
- Como isso ajuda no projeto do grupo?



Entregável

Formato: arquivo Markdown (.md)

Entrega: repositório do grupo no GitHub

/aula-10-testes-funcionais-automatizados.md

👉 Trabalho individual ou em grupo (até 4 integrantes)



Exemplo

<https://github.com/lucianozanuz/pbl-qualidade-software-2026-1/blob/main/pbl/aula-10-testes-funcionais-automatizados.md>



Avaliação (Rubrica – Unisenac-RS)



D — Não atingiu

- Não automatiza o fluxo corretamente
- Código não executa
- Sem refatoração



C — Parcial

- Teste executa parcialmente
- Uso direto do Codegen
- Baixa organização



B — Pleno

- Teste funcional completo
- Uso de Pytest
- Estrutura com POM básica



A — Excelência

- Código limpo e organizado
- Uso adequado de Page Object Model
- Boas práticas de seletores
- Teste robusto e confiável
- Análise crítica consistente



Dica final

Para obter conceito A, vocês devem:

- Não depender apenas do Codegen

- Melhorar seletores (evitar fragilidade)
- Organizar código com POM
- Criar testes claros e legíveis
- Pensar na manutenção do teste

👉 Mentalidade esperada:

“Se a interface mudar amanhã, meu teste ainda vai funcionar?”